

CliniqAI

Phase 1 — Core Agent & Drug-Alert System

Build · Learn · Win Toptal

Google Cloud Rapid Agent Hackathon · MongoDB Track · May 2026

1 — What We Built in Phase 1

Phase 1 is the **engine** of CliniqAI. No UI yet — just the three files that do all the real work: read a prescription photo, check for drug conflicts, and store everything in MongoDB.

Files created in Phase 1

```
cliniqai/
├── agent/
│   ├── __init__.py ← makes 'agent' a Python package
│   ├── server.py ← FastAPI backend (the brain)
│   └── tools/
│       ├── __init__.py
│       ├── vision_tool.py ← reads prescription photos
│       └── alert_tool.py ← catches drug conflicts
├── .env ← secret keys (never commit!)
├── requirements.txt ← Python dependencies
└── test_app.py ← 7 automated tests
```

2 — How It All Connects (ASCII Diagram)

Before any code — here is the complete flow from photo upload to alert:

```
Doctor uploads photo
|
▼
POST /process (server.py)
|
▼
vision_tool.py → Gemini reads image → returns JSON
|
▼
MongoDB: patient exists?
YES → update_one (push new visit)
NO → insert_one (new patient doc)
|
▼
alert_tool.py
Layer 1: Direct allergy check
Layer 2: Cross-allergy check
Layer 3: Drug-drug interaction check
|
▼
Return JSON to frontend: {patient, alerts, is_returning}
```

3 — Concepts (3-Layer Teaching)

Concept A — FastAPI

LAYER 1 — The Story

Imagine a reception desk at a clinic. Doctors walk up and say '*I have a photo*' or '*show me Ramesh's history*'. The receptionist understands them, does the work, and hands back an answer. FastAPI is that receptionist — it listens for requests from the outside world and routes them to the right function.

LAYER 2 — Technical

FastAPI is a Python web framework. You write a function and decorate it with `@app.post('/process')` — FastAPI automatically makes it available at that URL. It also generates interactive API docs at `/docs` automatically.

Key endpoints we built:

- GET `/health` — Is the server alive? Is MongoDB connected?
- POST `/process` — Upload image → extract → store → check alerts
- POST `/query` — Natural language search
- GET `/recent` — Last 10 patients
- POST `/test/process` — Test flow without Gemini API

LAYER 3 — Toptal Interview

Q: Why FastAPI over Flask or Django?

A: FastAPI has three advantages for CliniqAI: (1) async support — image processing and MongoDB calls can run concurrently; (2) automatic type validation via Pydantic — if someone POSTs the wrong JSON shape, FastAPI rejects it before my code even runs; (3) auto-generated /docs page — critical for demo. Flask has none of these by default. Django is too heavy for a single-purpose API.

FOLLOW-UP: What's the trade-off of FastAPI?

FOLLOW-UP ANSWER: Steeper learning curve than Flask for pure beginners. Also, it's ASGI-only — you need an ASGI server like uvicorn, not the classic gunicorn. In Cloud Run that's fine — we just run uvicorn.

Concept B — vision_tool.py (Gemini Vision)

LAYER 1 — The Story

Imagine hiring a medical secretary who speaks both Hindi and English, can read any doctor's handwriting (even the bad kind), and always fills in a standard form with the same fields every time. That is Gemini Vision. You give it a photo, it gives you clean structured data.

LAYER 2 — Technical

We send the image bytes + a structured prompt to Gemini's `generate_content()` method. The prompt tells Gemini to return **only** a JSON object — no extra words. Gemini sometimes wraps the JSON in markdown code blocks, so we strip those before parsing.

The prompt trick:

- Tell Gemini: *'Return ONLY the JSON. No preamble, no explanation.'*
- Strip ````json ... ```` markdown wrappers
- Wrap `json.loads()` in `try/except` — LLMs sometimes add comments

LAYER 3 — Toptal Interview

Q: How do you ensure consistent structured output from an LLM?

A: Three techniques: (1) prompt engineering — 'Return ONLY valid JSON matching this schema'; (2) post-processing — strip markdown wrappers, handle edge cases; (3) fallback — if `json.loads()` fails, return an error dict with the raw text for debugging. For production, I'd use Gemini's structured output mode or Pydantic validation.

FOLLOW-UP: Why not use GPT-4V?

FOLLOW-UP ANSWER: For this hackathon: cost and credits. Gemini 2.0 Flash is free on GCP credits, handles Hindi handwriting well, and is the stack the hackathon is built around. GPT-4V costs \$0.01 per image — at scale that matters.

Concept C — alert_tool.py (3-Layer Drug Checker)

LAYER 1 — The Story

Imagine a pharmacist who has memorised every drug family and every known interaction. A prescription arrives. Before handing over the medicine, they check three things: (1) Is the patient allergic to this drug family? (2) Even if not directly allergic — is there a related drug they react to? (3) Are two medicines in this bag known to be dangerous together? That is `alert_tool.py`.

LAYER 2 — Technical

Three independent checks run in sequence:

Layer 1 — Direct allergy: Patient has 'penicillin' allergy. New prescription has 'Amoxicillin'. Amoxicillin is in `ALLERGY_FAMILIES['penicillin']`. → HIGH alert.

Layer 2 — Cross-allergy: Penicillin allergy has ~10% cross-reactivity with cephalosporins. New prescription has 'Cefixime'. → MEDIUM alert.

Layer 3 — Drug-drug interaction: Patient currently on Warfarin. New prescription adds Aspirin. `DANGEROUS_COMBOS(['warfarin', 'aspirin'])` → HIGH alert.

All alerts are sorted: HIGH first. Output is a dict with `has_alerts`, `alert_count`, `high_severity`, `alerts[]`.

LAYER 3 — Toptal Interview

Q: Why rule-based instead of asking the LLM to check interactions?

A: Safety-critical systems must be deterministic. A known penicillin→amoxicillin allergy must trigger 100% of the time — not 97% of the time. LLMs are probabilistic. They can hallucinate. They can forget. A rule engine is a lookup table: it either matches or it doesn't. We use the LLM for extraction (where a near-miss is acceptable) and rules for safety (where it is not).

FOLLOW-UP: How would you scale the drug database?

FOLLOW-UP ANSWER: Replace the hardcoded dictionaries with a DrugBank or RxNorm API call. The function signature stays identical — callers don't change. This is the Open/Closed Principle: open for extension, closed for modification.

Concept D — MongoDB Patient Document Design

LAYER 1 — The Story

Imagine a physical file cabinet. Each patient has ONE folder. Inside that folder are ALL their visits — stacked on top of each other. One folder can have 2 pages, another can have 40. There's no fixed size. That is MongoDB. In SQL (like MySQL), you'd need a separate table for visits, joined with a patient ID. In MongoDB, one document = one patient = all their history in one place.

LAYER 2 — Technical

The patient document structure we use:

```
{ '_id': ObjectId,
  'patient_id': 'uuid-string',
  'name': 'Ramesh Gupta',
  'age': 45,
  'known_allergies': ['penicillin'],
  'conditions': ['Type 2 Diabetes'],
  'visits': [
    { 'date': '2026-05-16', 'doctor': 'Dr. Sharma',
      'medicines': [{name, dose, frequency, duration}],
      'tests': [], 'notes': '...' }
  ] }
```

Key MongoDB operations used:

- `insert_one()` — new patient
- `find_one({'name': name})` — check if returning
- `update_one(..., {'$push': {'visits': visit}})` — add visit
- `find({'conditions': {'$regex': ...}})` — search

LAYER 3 — Topical Interview

Q: Why MongoDB over PostgreSQL for patient records?

A: Patient records are schema-flexible — one patient has 2 medicines, another has 8. The visits array grows over time with no fixed length. MongoDB's document model fits this naturally. The trade-off is weaker ACID guarantees — but for a clinic app where we're appending visits and not doing financial transactions, that's acceptable. For billing, I'd use PostgreSQL.

FOLLOW-UP: How does \$push work?

FOLLOW-UP ANSWER: \$push is a MongoDB update operator that appends a value to an array field atomically. It's equivalent to `array.append()` in Python, but it happens inside MongoDB in one atomic operation — no risk of two doctors writing to the same patient record simultaneously and overwriting each other.

4 — API Keys Required

GOOGLE_API_KEY	Get from aistudio.google.com/app/apikey . Free. This lets Gemini read prescription images.
MONGODB_URI	Get from MongoDB Atlas → Connect → Drivers . Format: <code>mongodb+srv://user:pass@cluster.mongodb.net/cliniquai</code> . Free M0 cluster.

Without these keys the app still runs — it uses in-memory storage and returns a helpful error when you try to upload images. You can run all 7 tests without any API keys.

5 — Bug Discovery & Auto-Fix Report

Every bug found during Phase 1 build, with exact fix.

BUG #1 — Server crashes on startup [CRITICAL]

WHAT HAPPENED

When the .env file has placeholder values (youruser:yourpassword), PyMongo does a DNS lookup immediately at import time and throws `ConfigurationError: The DNS query name does not exist`. The server exits before any request is handled.

BEFORE

```
client = MongoClient(MONGODB_URI) # Line 32 – runs at import time
db = client['cliniqai'] # Crashes here if URI is fake
patients_collection = db['patients']
```

AFTER

```
def get_db():
    global client, db, patients_collection
    if patients_collection is not None:
        return True # Already connected
    if not MONGODB_URI or 'youruser' in MONGODB_URI:
        return False # Not configured – use in-memory
    try:
        client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=5000)
        client.admin.command('ping')
    ...
```

ROOT CAUSE: Eager vs lazy initialization. The fix moves connection to first use.

BUG #2 — Non-image upload returns HTTP 500 [MEDIUM]

WHAT HAPPENED

Uploading a .txt or corrupted file caused `PIL.UnidentifiedImageError` — unhandled exception → server returned a raw 500 error with no useful message to the frontend.

BEFORE

```
image = Image.open(io.BytesIO(image_bytes)) # Crashes on non-images
```

AFTER

```
try:
    image = Image.open(io.BytesIO(image_bytes))
except Exception as e:
    return {'error': f'Could not read image file: {str(e)}. Please upload a valid JPG or PNG.'}
```

BUG #3 — No API key guard before calling Gemini [MEDIUM]

WHAT HAPPENED

With a placeholder key, `genai.configure(api_key='your_google_api_key_here')` doesn't fail immediately — it fails deep inside Google's library with a cryptic authentication error. Hard to debug.

AFTER

```
api_key = os.getenv('GOOGLE_API_KEY')
if not api_key or 'your_google' in api_key:
    return {'error': 'GOOGLE_API_KEY not configured. Set it in .env file.'}
```

BUG #4 — Regex injection in MongoDB query [LOW]

WHAT HAPPENED

A user query containing regex special characters like `test (patient)` would pass directly into `{'$regex': query}` — MongoDB would throw a regex parse error.

AFTER

```
import re
safe_query = re.escape(query) # Escapes (, ), [, ], ., * etc.
patients = collection.find({'name': {'$regex': safe_query, '$options': 'i'}})
```

BUG #5 — Pinned dependency versions conflicted with google-adk [LOW]

WHAT HAPPENED

`fastapi==0.115.0` conflicted with `google-adk` which requires `fastapi>=0.124.1`. pip reported dependency conflicts.

AFTER

```
# requirements.txt — use >= instead of ==
fastapi>=0.124.1
uvicorn>=0.34
python-dotenv>=1.0.1
```

6 — Test Results (All 7 Passed)

TEST 1	Health check — server running, MongoDB status shown
TEST 2	Allergy alert: penicillin allergy + amoxicillin → HIGH alert fired
TEST 3	No alerts: safe medicines + no allergies → no alerts
TEST 4	Drug interaction: warfarin + aspirin → HIGH bleeding risk alert
TEST 5	Cross-allergy: penicillin allergy + cefixime → MEDIUM cross-allergy warning
TEST 6	Query endpoint: 'how many patients' → correct count returned

TEST 7	Recent patients: empty list returned correctly from in-memory store
--------	---

7 — Why Not the Alternatives

We used	FastAPI
Could have used	Flask, Django
Why FastAPI wins	Async support + auto docs + Pydantic validation. Flask has none by default. Django is overkill.
We used	Rule-based alert_tool
Could have used	Ask Gemini to check interactions
Why rules win	Deterministic. A known allergy must trigger 100% of the time. LLMs are probabilistic — unsafe for life-critical checks.
We used	In-memory fallback store
Could have used	Crash with error if no MongoDB
Why fallback wins	Lets you develop and test the full flow locally without any cloud account. Faster dev loop.
We used	pymongo (direct)
Could have used	MongoDB MCP Server
Why pymongo now	Phase 1 is about getting core logic working cleanly. MCP adds complexity — it requires a running Node.js process. We add it in Phase 3 when the ADK agent needs it.

8 — How to Run Phase 1

```
# 1. Install dependencies
cd cliniqai
pip install -r requirements.txt

# 2. Fill in your API keys
# Edit .env: set GOOGLE_API_KEY and MONGODB_URI

# 3. Start the server
python -m uvicorn agent.server:app --host 127.0.0.1 --port 8000 --reload

# 4. Run all 7 tests (in a second terminal)
python test_app.py

# 5. Open interactive API docs
http://127.0.0.1:8000/docs
```